

	Colegio Universitario de Cartago	
	BIG DATA (BD)	

II Cuatrimestre 2026
BD-143 Programación II
Profesor Osvaldo González Chaves
Proyecto II

Descripción del proyecto

Desarrollar un sistema, en Python y con Programación Orientada a Objetos, que ingeste datos de partidos de la Copa Mundial de la FIFA desde un archivo CSV, los procese en memoria mediante estructuras y clases bien definidas, y permita consultas, análisis exploratorio (EDA) y visualización de gráficos o de manera interactiva mediante un dashboard.

Objetivos

- **Ingesta:** Cargar el archivo CSV con partidos de la Copa Mundial de la FIFA (1930–2022).
- **Persistencia en archivos:** Guardar el conjunto de datos procesado en formato CSV para su reutilización posterior, sin uso de bases de datos.
- **EDA:** Proveer métodos orientados a objetos y un notebook con análisis descriptivo.
- **Visualización:** Crear gráficos clave (que cuenten una historia) y/o un dashboard (Streamlit) para comunicar hallazgos.

Rubros de calificación

Criterio	Porcentaje
Programación Orientada a Objetos (POO)	40%
Ingesta y manipulación de datos con Pandas	25%
Análisis exploratorio de datos (EDA)	20%
Visualización de datos (gráficos, tendencias, comparaciones, etc.)	15%
<i>Puntos extra: Dashboard interactivo (ej. con Streamlit)</i>	2%

Total: 100%

Dataset

Se utilizará el dataset público “International football results from 1872 to 2026” mantenido por martj42 en GitHub, el cual incluye más de 49,000 partidos internacionales y contiene la columna tournament que permite filtrar exclusivamente los partidos de la Copa Mundial de la FIFA.

URL del CSV (descarga directa):

https://raw.githubusercontent.com/martj42/international_results/master/results.csv

Repositorio oficial: https://github.com/martj42/international_results

Columnas relevantes del CSV:

- `date` — fecha del partido (YYYY-MM-DD).
- `home_team` — equipo local.
- `away_team` — equipo visitante.
- `home_score` — goles del equipo local.
- `away_score` — goles del equipo visitante.
- `tournament` — torneo (filtrar por "FIFA World Cup").
- `city` — ciudad sede del partido.
- `country` — país sede del partido.
- `neutral` — indica si el partido se jugó en cancha neutral (True/False).

Requerimientos funcionales del proyecto World Cup Insights

1. **Descarga e ingesta de datos:** descargar el archivo `results.csv` desde la URL pública y filtrar únicamente los partidos cuyo `tournament` sea "FIFA World Cup". Guardar el subconjunto resultante en `data/raw/partidos-mundial.csv` mediante la Clase `CargadorDatos` pueden usar los atributos que sean necesarios.
2. **Consultas sobre los datos:** exponer métodos de solo lectura en la Clase `GestorPartidos` que permitan extraer información del `DataFrame`, como por ejemplo: `get_partido(id)`, `get_por_equipo(equipo)`, `get_por_anio(anio)`, `get_por_sede(pais)`, `ventaja_local()`. Cree todos los necesarios que considere.
3. **Persistencia en archivos:** guardar el dataset procesado (limpio, con columnas derivadas) en CSV y/o JSON dentro de `data/processed/` mediante un método de la Clase `CargadorDatos`. No se requiere base de datos relacional para esta entrega.
4. **Procesamiento EDA.** La Clase `ProcesadorEDA` debe generar al menos:
 - Limpieza de datos (`limpieza_datos`)
 - Resumen descriptivo (`resumen_descriptivo`)
 - Matriz de correlación (`matriz_correlacion`)
 - Columnas derivadas: año, total de goles, diferencia de goles, ganador (Local/Visitante/Empate).
 - Puede crear más métodos que considere (detección de outliers, agrupaciones por edición, etc.).
5. **Visualización estática e interactiva:** la Clase `Visualizador` creará histogramas, scatter plots, heatmaps, gráficos de barras, etc. utilizando Matplotlib / Seaborn / Plotly. Cada visualización debe contar una historia; sea creativo y analítico, obtenga estadísticas interesantes en los datos (por ejemplo: ¿el país sede gana más?, ¿en qué Mundial se metieron más goles por partido?, ¿qué selección tiene la mejor diferencia de goles histórica? etc).

Estructura de carpetas del proyecto

En PyCharm, utilice el .zip “world_cup_insights” y cree las carpetas y archivos necesarios según el siguiente esquema:

```
world_cup_insights/
├── src/                    # Código fuente principal
│   ├── ingesta/           # Carga del CSV y persistencia (Clase CargadorDatos)
│   ├── gestor/            # Consultas sobre el DataFrame (Clase GestorPartidos)
│   ├── eda/               # Exploración y estadísticas descriptivas (Clase ProcesadorEDA)
│   ├── visualizacion/     # Visualización de datos (Clase Visualizador)
│   ├── helpers/           # Funciones auxiliares reutilizables (Clase Utilidades)
│   └── main.py             # Punto de entrada del proyecto
├── notebooks/             # Jupyter notebooks para desarrollo y presentación
│   ├── 01_EDA.ipynb
│   └── 02_Visualizacion.ipynb
├── data/
│   ├── raw/               # CSV original descargado (partidos-mundial.csv)
│   └── processed/         # CSV/JSON ya procesado por el sistema
├── dashboard/             # Streamlit (opcional)
│   └── app.py              # Ejecutar con: streamlit run dashboard/app.py
└── README.md              # Documente el proyecto en un markdown
```

Detalle de los módulos y clases

- `ingesta/` → **Clase CargadorDatos**: descarga el CSV desde la URL pública, filtra los partidos de la Copa Mundial, valida los datos y guarda los archivos en `/data/raw` y `/data/processed`.
- `gestor/` → **Clase GestorPartidos**: expone métodos de solo lectura para consultar el DataFrame (por equipo, año, sede, etc.).
- `eda/` → **Clase ProcesadorEDA**: realiza análisis estadístico, limpieza y exploración inicial de los datos.
- `visualizacion/` → **Clase Visualizador**: crea gráficos (líneas, barras, heatmaps, etc.). Sea creativo: cada gráfico debe contar una historia.
- `helpers/` → **Clase Utilidades**: contiene funciones auxiliares reutilizables para validaciones, formateo, etc. (opcional).
- **Notebook 01_EDA.ipynb**: invoca las clases CargadorDatos, GestorPartidos y ProcesadorEDA.
- **Notebook 02_Visualizacion.ipynb**: invoca la clase Visualizador.

Puntos extra

Como complemento, se propone generar un dashboard interactivo en una plataforma gratuita como Streamlit, para visualizar resultados del análisis exploratorio de manera sencilla y atractiva.

Entregables

6. Repositorio Git con el código fuente y los notebooks.
7. Archivo CSV procesado en /data/processed/ generado por el propio sistema.
8. README.md con instrucciones para ejecutar el proyecto.

Indicaciones

9. Realizarlo en parejas.
10. Fecha de entrega: 14-07-2026.
11. Debe correr todo en el momento de la revisión; asegúrese que esté listo cuando el profesor lo llame a revisión. No habrá ampliación de tiempo para ajustes en el código.
12. Debe estar el código en el repositorio antes de la revisión.